Banking Transaction Analysis System
File, Function and Role Review for Project Review

This document lists every project file, its responsible role, a short description, and the functions/classes it contains. It is intended for review and viva/demo preparation.
Roles are based on docs/team_roles.md. Shared files are owned by all members.

Role-wise file count
- Member 1 - Backend + Integration: 14 files
- Member 2 - Database: 5 files
- Member 3 - Data Analysis: 7 files
- Member 4 - UI + Visualization: 17 files
- Shared (all members): 9 files

Contents

- Member 1 - Backend + Integration
- Member 2 - Database
- Member 3 - Data Analysis
- Member 4 - UI + Visualization
- Shared (all members)

Member 1 - Backend + Integration

.streamlit/config.toml
Owner: Member 1 - Backend + Integration
Purpose: Configuration file.
- [theme]
- base = "light"
- primaryColor = "#0F766E"
- backgroundColor = "#F4F7FB"
- secondaryBackgroundColor = "#FFFFFF"
- textColor = "#0F172A"
- font = "sans serif"
- [server]
- headless = false
- [browser]
- gatherUsageStats = false

backend/__init__.py
Owner: Member 1 - Backend + Integration
No top-level functions or classes (module-level code only).

backend/account_service.py
Owner: Member 1 - Backend + Integration
Purpose: Account CRUD (MEMBER 1).
Imports: backend, datetime, json, utils
Functions:
- create_account(customer_id, account_type, opening_balance, account_number) - Open a new account for an existing customer and return the new account_id.
- get_account(account_id)
- get_all_accounts(status) - All accounts, or only 'ACTIVE' / 'INACTIVE' ones.
- get_accounts_by_customer(customer_id)

- count_accounts()
- search_accounts(keyword) - Search by account number, customer name, account type or status.
- update_account(account_id, account_type, status) - Change the account type and/or status. The balance can only change via transactions.
- deactivate_account(account_id) - Accounts are deactivated, not deleted, so the transaction history is kept.
- activate_account(account_id)
- count_transactions(account_id) - Number of transactions recorded for an account (0 is safe to delete directly).
- delete_account(account_id, delete_transactions) - Move an account to the recycle bin and delete it (it can be restored later).
- _transactions_snapshot(cursor, account_id) - JSON copy of an account's transactions, stored in the recycle bin for restore.
- get_deleted_accounts(limit) - Recently deleted accounts (newest first) for the 'Recently Deleted' tab.
- restore_account(archive_id) - Put a deleted account back, together with its transactions.
- purge_deleted_account(archive_id) - Remove one entry from the recycle bin permanently (the account cannot come back).

backend/balance_service.py

Owner: Member 1 - Backend + Integration

Purpose: Balance calculation (MEMBER 1).

Imports: backend, utils

Functions:

- calculate_balance(account_id) - Balance worked out from the transactions (deposits - withdrawals).
- get_account_balance(account_id) - Balance stored in the accounts table.
- recalculate_balance_in_cursor(cursor, account_id) - Recalculate the balance from transactions and save it to the accounts table.
- get_total_balance() - Sum of the balances of all ACTIVE accounts.
- find_balance_mismatches() - List accounts whose stored balance differs from the transaction total (data check).

backend/customer_service.py

Owner: Member 1 - Backend + Integration

Purpose: Customer CRUD (MEMBER 1).

Imports: backend, utils

Functions:

- create_customer(name, phone, email, address) - Add a customer and return the new customer_id.
- get_customer(customer_id) - Return one customer as a dict, or None if not found.
- get_all_customers()
- count_customers()
- search_customers(keyword) - Search by name, phone, email or customer ID.
- update_customer(customer_id, name, phone, email, address) - Update only the fields that are given (not None). Returns True if a row changed.
- delete_customer(customer_id) - Delete a customer. Customers who still own accounts cannot be deleted.

backend/db_connection.py

Owner: Member 1 - Backend + Integration

Purpose: Database connection helpers (MEMBER 1).

Imports: contextlib, dotenv, logging, mysql, os, pathlib

Classes:

- DatabaseError - Friendly database error. str(error) is safe to show to the user.
- DatabaseConnectionError - MySQL is not reachable or the login details are wrong.
- DuplicateEntryError - A UNIQUE value (phone, email, account number...) already exists.
- RelatedRecordsError - The row is linked to other rows (foreign key), so it cannot be changed/deleted.

Functions:

- _translate_error(error) - Convert a raw MySQL error into one of our friendly errors.
- get_connection() - Open a new MySQL connection using environment variables.
- get_cursor(commit) - Yield a dictionary cursor. Rolls back automatically on any error.
- fetch_all(sql, params) - Run a SELECT and return a list of dictionaries.
- fetch_one(sql, params) - Run a SELECT and return one dictionary (or None).
- execute(sql, params) - Run INSERT/UPDATE/DELETE, commit, and return (last_insert_id, affected_rows).
- check_connection() - Return True if the database can be reached (used at dashboard start-up).

backend/transaction_service.py

Owner: Member 1 - Backend + Integration

Purpose: Transaction operations: deposit, withdrawal, CRUD, search and filters (MEMBER 1).

Imports: backend, datetime, utils

Classes:

- InsufficientBalanceError - Withdrawal is bigger than the money available in the account.

Functions:

- apply_transaction(cursor, account_id, transaction_type, amount, transaction_date, category, description, reference_number) - Core deposit/withdrawal logic. Works on an open cursor so that callers
- add_transaction(account_id, transaction_type, amount, transaction_date, category, description, reference_number) - Validate the input, then record a deposit or withdrawal. Returns transaction_id.
- deposit(account_id, amount, category, description, transaction_date)
- withdraw(account_id, amount, category, description, transaction_date)
- get_transaction(transaction_id)
- get_transactions(account_id, start_date, end_date, transaction_type, category, min_amount, max_amount, keyword, limit) - View / filter / search transactions (newest first). All arguments are optional.
- search_transactions(keyword) - Search description, reference number, category or account number.
- update_transaction(transaction_id, transaction_type, amount, transaction_date, category, description) - Edit a transaction (the account cannot be changed). The balance is recalculated
- delete_transaction(transaction_id) - Delete a transaction and fix the balance (rejected if the balance would go negative).
- get_categories() - Distinct categories currently used in the transactions table.
- get_date_range() - (earliest_date, latest_date) of all transactions, or None if there are none.

requirements.txt

Owner: Member 1 - Backend + Integration

Purpose: Text/dependency file.

- streamlit>=1.36.0
- pandas>=2.0.0
- numpy>=1.24.0
- plotly>=5.18.0
- mysql-connector-python>=8.0.33
- python-dotenv>=1.0.0
- pytest>=7.4.0

run.bat

Owner: Member 1 - Backend + Integration

Purpose: Launcher script.

Commands:

- @echo off
- REM ==
- REM Banking Transaction Analysis - launcher
- REM Double-click this file to start the dashboard.
- REM Press Ctrl+C in this window (or close it) to stop.
- REM ==
- title Banking Transaction Analysis
- cd /d "%~dp0"
- set "PYTHON=python"
- if exist ".venv\Scripts\python.exe" set "PYTHON=.venv\Scripts\python.exe"
- if exist "venv\Scripts\python.exe" set "PYTHON=venv\Scripts\python.exe"
- where "%PYTHON%" >nul 2>nul

tests/test_transactions.py

Owner: Member 1 - Backend + Integration

Purpose: Transaction logic tests: deposit, withdrawal, balance rules (needs MySQL).

Imports: backend, datetime, decimal, pytest, utils

Functions:

- test_opening_balance_is_a_deposit(account_id)
- test_deposit_increases_balance(account_id)
- test_withdrawal_decreases_balance(account_id)
- test_insufficient_balance_rejected(account_id)
- test_invalid_amount_rejected(account_id, amount)
- test_invalid_type_account_and_date_rejected(account_id)
- test_inactive_account_rejected(account_id)
- test_balance_calculation_matches_stored_balance(account_id)
- test_update_and_delete_recalculate_balance(account_id)
- test_update_that_makes_balance_negative_is_rejected(account_id)
- test_filter_and_search(account_id)
- test_sql_injection_text_is_harmless(account_id)

tests/test_validators.py

Owner: Member 1 - Backend + Integration

Purpose: Tests for utils/validators.py (no database needed).

Imports: datetime, decimal, pytest, utils

Functions:

- test_valid_amount_is_rounded_decimal()
- test_invalid_amounts_rejected(bad)
- test_transaction_type()
- test_date_validation()
- test_phone_and_email()
- test_name_account_number_and_id()
- test_account_type_and_status()
- test_indian_currency_format()

utils/__init__.py

Owner: Member 1 - Backend + Integration

No top-level functions or classes (module-level code only).

utils/helpers.py

Owner: Member 1 - Backend + Integration

Purpose: Small helper functions and constants used in many places.

Imports: datetime, decimal, uuid

Functions:

- format_inr(amount) - Format a number the Indian way: 1234567.5 -> 'Rs.12,34,567.50'.
- generate_reference_number() - Unique reference such as TXN20260325A1B2C3D4.
- generate_account_number() - Random 12-digit account number (the DB UNIQUE key rejects the rare duplicate).

utils/validators.py

Owner: Member 1 - Backend + Integration

Purpose: Input validators (shared by backend, tests and dashboard).

Imports: datetime, decimal, re

Classes:

- ValidationError - Raised when user input or a business rule is not acceptable.

Functions:

- validate_id(value, field_name) - Return a positive integer ID.
- validate_name(name)
- validate_phone(phone) - Accepts '9876543210', '+91 98765 43210', '98765-43210'. Returns 10 digits.
- validate_email(email)
- validate_address(address)
- validate_account_number(account_number)
- validate_account_type(account_type)
- validate_account_status(status)
- validate_transaction_type(transaction_type)
- validate_amount(amount, allow_zero) - Return the amount as a Decimal rounded to 2 places.
- validate_date(value, allow_future) - Return a datetime. Accepts datetime, date or 'YYYY-MM-DD[HH:MM[:SS]]' text.
- validate_category(category)
- validate_description(description)

Member 2 - Database

database/README.md

Owner: Member 2 - Database

Purpose: Markdown documentation.

Sections:

- Database (Member 2)
- Run it in SQLyog
- Tables

database/queries.sql

Owner: Member 2 - Database

Purpose: SQL script (13 statements).

Statements:

- -- ==
- -- 1. All customers with the number of accounts they own
- -- 2. Accounts with owner name and balance
- -- 3. Latest 20 transactions
- -- 4. Total deposits and withdrawals

- -- 5. Balance of each account calculated from its transactions
- -- 6. Monthly expenses
- -- 7. Category-wise expenses
- -- 8. Largest 10 transactions
- -- 9. Search transactions by keyword
- -- 10. Withdrawals above 50,000 (simple large-transaction check)
- -- 11. Accounts with more than 5 transactions on one day (busy days)
- -- 12. Average transaction amount per account

database/sample_data.sql

Owner: Member 2 - Database

Purpose: SQL script (6 statements).

Statements:

- -- ==
- INSERT INTO customers (name, phone, email, address) VALUES
- -- balance starts at 0
- it is recalculated from transactions at the bottom
- INSERT INTO transactions (account_id, transaction_date, transaction_type, amount, category, description, reference_number) VALUES
- -- Recalculate every account balance from its transactions

database/schema.sql

Owner: Member 2 - Database

Purpose: SQL script (10 statements).

Statements:

- -- ==
- USE banking_db
- DROP TABLE IF EXISTS deleted_accounts
- DROP TABLE IF EXISTS transactions
- DROP TABLE IF EXISTS accounts
- DROP TABLE IF EXISTS customers
- -- --
- -- --
- -- --
- -- --

tests/test_database.py

Owner: Member 2 - Database

Purpose: Database tests: connection, tables and customer/account CRUD (needs MySQL).

Imports: backend, pytest, tests, utils

Functions:

- test_database_connection()
- test_tables_exist()
- test_customer_create_read_update_search_delete()
- test_duplicate_customer_rejected(customer_id)
- test_invalid_customer_input_rejected(require_database)
- test_account_creation_and_duplicate_number(customer_id)
- test_account_for_missing_customer_rejected(require_database)
- test_deactivate_account_and_customer_delete_blocked(account_id, customer_id)
- test_delete_account_requires_history_confirmation(account_id) - An account with transactions is only deleted when history deletion is confirmed.
- test_deleted_account_is_archived_and_can_be_restored(account_id) - Deleting moves the account to the recycle bin; restoring brings back its history.

- test_purge_deleted_account_removes_it_permanently(account_id)

Member 3 - Data Analysis

analysis/__init__.py

Owner: Member 3 - Data Analysis

No top-level functions or classes (module-level code only).

analysis/monthly_analysis.py

Owner: Member 3 - Data Analysis

Purpose: Month-wise analysis (MEMBER 3). Months are shown as 'YYYY-MM'.

Imports: pandas

Functions:

- _with_month(df)
- _monthly_total(df, transaction_type, column_name)
- monthly_expenses(df) - Total withdrawals per month -> columns: month, expense.
- monthly_income(df) - Total deposits per month -> columns: month, income.
- monthly_summary(df) - Income, expense and net for every month -> columns: month, income, expense, net.

analysis/report_generator.py

Owner: Member 3 - Data Analysis

Purpose: Build report tables and CSV bytes for download (MEMBER 3).

Imports: analysis, pandas

Functions:

- transaction_report(df) - All given transactions, newest first.
- monthly_spending_report(df)
- category_spending_report(df)
- unusual_transaction_report(df)
- build_report(report_name, df) - Return the DataFrame for one of REPORT_NAMES.
- to_csv_bytes(report_df) - CSV as bytes with a BOM so Excel shows the Rs. sign correctly.
- report_file_name(report_name)

analysis/spending_analysis.py

Owner: Member 3 - Data Analysis

Purpose: Category-wise spending and income vs expense (MEMBER 3).

Imports: analysis, pandas

Functions:

- category_expenses(df) - Total withdrawals per category (largest first) -> columns: category, expense, transactions.
- top_spending_categories(df, top_n)
- income_vs_expense(df) - Dictionary with total_income, total_expense and net_balance.
- account_spending(df) - Total income and expense of each account -> useful for comparing accounts.

analysis/transaction_analysis.py

Owner: Member 3 - Data Analysis

Purpose: Core transaction analysis with Pandas / NumPy (MEMBER 3).

Imports: numpy, pandas

Functions:

- prepare_dataframe(records) - Turn a list of dicts (or a DataFrame) into a clean DataFrame with correct data types.
- load_transactions() - Fetch transactions from MySQL (optionally filtered) as a DataFrame.

- total_deposits(df)
- total_withdrawals(df)
- calculate_balance(df) - Deposits minus withdrawals of the given transactions.
- average_transaction(df)
- largest_transaction(df) - Return the largest transaction as a dict, or None if there is no data.
- transaction_statistics(df) - count, sum, mean, median, min, max for all / deposits / withdrawals (one row each).

analysis/unusual_transactions.py

Owner: Member 3 - Data Analysis

Purpose: Unusual transaction detection (MEMBER 3).

Imports: numpy, pandas

Functions:

- _rule_high_vs_normal(df, z_threshold)
- _rule_high_amount(df, amount_threshold)
- _rule_frequency(df, max_transactions, window_minutes)
- unusual_transactions(df, z_threshold, amount_threshold, max_transactions, window_minutes) - Return the flagged transactions with a ‘reason‘ column (empty DataFrame if none).

tests/test_analysis.py

Owner: Member 3 - Data Analysis

Purpose: Analysis tests with a small hand-made DataFrame (no database needed).

Imports: analysis, pandas

Functions:

- make_df()
- make_history_df() - 20 small daily withdrawals plus one huge one (id 999).
- test_totals_and_balance()
- test_average_and_largest()
- test_statistics()
- test_monthly_analysis()
- test_category_analysis_and_income_vs_expense()
- test_empty_dataframe_does_not_crash()
- test_unusual_amount_much_higher_than_normal()
- test_unusual_amount_threshold_is_configurable()
- test_unusual_frequency_rule()
- test_csv_export_is_bytes()

Member 4 - UI + Visualization

dashboard/__init__.py

Owner: Member 4 - UI + Visualization

No top-level functions or classes (module-level code only).

dashboard/app.py

Owner: Member 4 - UI + Visualization

Purpose: Streamlit entry point (MEMBER 4). Run from the project root with:

Imports: backend, dashboard, pathlib, streamlit, sys

No top-level functions or classes (module-level code only).

dashboard/assets/style.css

Owner: Member 4 - UI + Visualization

Purpose: Stylesheet (123 selectors).

Selectors:

- :root
- html, body, [class*="css"], .stApp, button, input, textarea, select
- .stApp
- .block-container
- header[data-testid="stHeader"]
- #MainMenu, footer
- h1, h2, h3, h4
- h2, h3
- section[data-testid="stSidebar"]
- section[data-testid="stSidebar"] .block-container
- [data-testid="stSidebarNav"] a
- [data-testid="stSidebarNav"] a:hover
- [data-testid="stSidebarNav"] a[aria-current="page"]
- [data-testid="stSidebarNav"] a[aria-current="page"] span
- .brand
- .brand-logo
- .brand-name
- .brand-tag
- .hero
- .hero::after
- .hero-title
- .hero-sub
- .kpi
- .kpi:hover
- .kpi::before
- .kpi-top
- .kpi-label
- .kpi-icon
- .kpi-value
- .kpi-note
- .kpi.green
- .kpi.red
- .kpi.blue
- .kpi.amber
- .kpi.teal
- [data-testid="stMetric"]
- [data-testid="stMetricLabel"]
- [data-testid="stMetricValue"]
- .section-title
- .section-title::after
- .badge
- .badge.green
- .badge.red
- .badge.gray
- [data-testid="stPlotlyChart"], [data-testid="stDataFrame"]
- [data-testid="stForm"]
- .stTabs [data-baseweb="tab-list"]
- .stTabs [data-baseweb="tab"]
- .stTabs [aria-selected="true"]
- .stTabs [data-baseweb="tab-highlight"], .stTabs [data-baseweb="tab-border"]
- .stButton > button, .stDownloadButton > button, [data-testid="stFormSubmitButton"] > button
- .stButton > button:hover, .stDownloadButton > button:hover, [data-testid="stFormSubmitButton"] >

button:hover
- .stButton > button:disabled, .stDownloadButton > button:disabled
- div[data-baseweb="input"], div[data-baseweb="select"] > div, div[data-baseweb="base-input"]
- div[data-baseweb="input"]:focus-within, div[data-baseweb="select"] > div:focus-within
- [data-testid="stAlert"]
- [data-testid="stExpander"]
- hr
- ::-webkit-scrollbar
- ::-webkit-scrollbar-thumb

... and 63 more selectors.

dashboard/auth.py

Owner: Member 4 - UI + Visualization

Purpose: Demo authentication helpers for the Banking Transaction Analysis app.

Imports: __future__, hmac, streamlit

Functions:

- is_authenticated() - Return True when a demo user has signed in for this session.
- current_user()
- logout()
- attempt_login(username, password, selected_role) - Validate a demo account and start an authenticated session.

dashboard/components/__init__.py

Owner: Member 4 - UI + Visualization

No top-level functions or classes (module-level code only).

dashboard/components/charts.py

Owner: Member 4 - UI + Visualization

Purpose: Plotly chart builders (MEMBER 4). Each function takes a DataFrame and returns a figure.

Imports: plotly

Functions:

- _style(figure) - Apply the shared look (fonts, transparent background, soft grid) to every chart.
- _empty_figure(title)
- monthly_expense_chart(monthly_expense_df) - Bar chart of expenses per month (columns: month, expense).
- income_vs_expense_chart(summary_df) - Grouped bars: income and expense for every month (columns: month, income, expense).
- category_pie_chart(category_df) - Donut chart of spending per category (columns: category, expense).
- category_bar_chart(category_df) - Horizontal bar chart of spending per category.
- monthly_net_chart(summary_df) - Line chart of the net amount (income - expense) per month.
- account_comparison_chart(account_df) - Income vs expense of every account (columns: account_number, income, expense).
- amount_histogram(df) - Histogram showing how transaction amounts are spread.

dashboard/components/tables.py

Owner: Member 4 - UI + Visualization

Purpose: Shared Streamlit widgets: tables, filters, error handling (MEMBER 4).

Imports: analysis, backend, datetime, pandas, streamlit, utils

Functions:

- run_safely(function) - Call a backend function; show a friendly message instead of crashing.
- records_to_df(records) - List of dicts from the backend -> DataFrame (Decimal money values

become floats).
- show_table(df, money_columns) - Show a DataFrame as a nice full-width table with Rs. formatting.
- show_chart(figure, key) - Draw a Plotly figure at full width (works on old and new Streamlit versions).
- show_money(label, amount)
- account_options(only_active) - {label: account_id} for dropdowns.
- transaction_filters(container, key_prefix) - Draw Account / Date range / Type / Category filters inside ‘container‘
- unusual_settings(container, key_prefix) - Configurable thresholds for the unusual-transaction rules. Returns keyword arguments.
- show_unusual_transactions(flagged_df) - Show alerts for flagged transactions (used on the dashboard and analysis page).

dashboard/landing.py

Owner: Member 4 - UI + Visualization

Purpose: Public landing page shown before the demo login portal.

Imports: __future__, dashboard, html, streamlit

Functions:

- _stats_html()
- _feature_card(icon, title, text)
- render() - Render the public landing page with a button to the separate login portal.

dashboard/login.py

Owner: Member 4 - UI + Visualization

Purpose: Separate demo login portal for the Banking Transaction Analysis app.

Imports: __future__, dashboard, streamlit

Functions:

- _tame_password_manager() - Stop browsers from offering to generate a new strong password on login.
- render() - Render a plain username/password login page.

dashboard/pages/__init__.py

Owner: Member 4 - UI + Visualization

No top-level functions or classes (module-level code only).

dashboard/pages/accounts.py

Owner: Member 4 - UI + Visualization

Purpose: Account management page (MEMBER 4).

Imports: backend, dashboard, pandas, streamlit, utils

Functions:

- render()

dashboard/pages/analysis.py

Owner: Member 4 - UI + Visualization

Purpose: Analysis page: statistics, monthly, category, income vs expense, unusual (MEMBER 4).

Imports: analysis, dashboard, streamlit, utils

Functions:

- render()

dashboard/pages/customers.py

Owner: Member 4 - UI + Visualization

Purpose: Customer management page (MEMBER 4).

Imports: backend, dashboard, streamlit

Functions:

- _customer_label(customer)
- render()

dashboard/pages/dashboard.py

Owner: Member 4 - UI + Visualization

Purpose: Main dashboard page: KPI cards, charts, recent transactions, alerts (MEMBER 4).

Imports: analysis, backend, dashboard, streamlit, utils

Functions:

- render()

dashboard/pages/reports.py

Owner: Member 4 - UI + Visualization

Purpose: Report page: preview and CSV download (MEMBER 4).

Imports: analysis, dashboard, streamlit

Functions:

- render()

dashboard/pages/transactions.py

Owner: Member 4 - UI + Visualization

Purpose: Transaction management page: view/search/filter, add, edit, delete (MEMBER 4).

Imports: backend, dashboard, datetime, streamlit, utils

Functions:

- _view_tab()
- _add_tab()
- _edit_tab()
- render()

dashboard/theme.py

Owner: Member 4 - UI + Visualization

Purpose: Theme helpers: loads the custom CSS and renders small HTML components.

Imports: html, pathlib, streamlit

Functions:

- apply_theme() - Inject the custom stylesheet and the sidebar brand block. Call once in app.py.
- page_header(icon, title, subtitle) - Gradient hero banner shown at the top of every page.
- section_title(icon, text)
- kpi_card(label, value, icon, tone, note) - HTML KPI card. tone: teal | green | red | blue | amber.

Shared (all members)

.gitignore

Owner: Shared (all members)

Purpose: file asset.

- # secrets
- .env
- # python
- venv/
- .venv/
- __pycache__/
- *.pyc
- .pytest_cache/

README.md

Owner: Shared (all members)

Purpose: Markdown documentation.

Sections:

- Banking Transaction Analysis System
- Features
- Technology stack
- Architecture
- Database structure
- Folder structure
- Team responsibilities
- Installation
- 1. MySQL
- 2. Create the database with SQLyog
- 3. Python environment
- Windows (Command Prompt)
- Windows (PowerShell)
- macOS / Linux
- 4. Configure '.env'
- Windows
- macOS / Linux
- How to run
- Demo login accounts
- How to use the dashboard
- Unusual transaction detection (rules)
- Git workflow
- open a Pull Request on GitHub -> review -> merge into main
- Testing
- Future improvements
- Limitations
- Theme / customising the look

docs/architecture.md

Owner: Shared (all members)

Purpose: Markdown documentation.

Sections:

- Architecture
- Rules
- Key design decisions

docs/database_design.md

Owner: Shared (all members)

Purpose: Markdown documentation.

Sections:

- Database design (banking_db)
- Entities and relationships
- customers
- accounts
- transactions
- Normalization

docs/file_function_review.pdf

Owner: Shared (all members)
Purpose: .pdf asset.
- %PDF-1.4
- 6 0 obj
- << /Length 3104 >>
- stream
- BT
- /F1 10 Tf
- 1 0 0 1 50 742 Tm
- 13 TL

docs/project_workflow.md
Owner: Shared (all members)
Purpose: Markdown documentation.
Sections:
- Project workflow (Git + GitHub)
- Branches
- One-time setup (Member 1)
- Everyone else
- Keeping your branch up to date
- How Member 1 integrates the branches

docs/team_roles.md
Owner: Shared (all members)
Purpose: Markdown documentation.
Sections:
- Team roles
- Interfaces between members

tests/__init__.py
Owner: Shared (all members)
No top-level functions or classes (module-level code only).

tests/conftest.py
Owner: Shared (all members)
Purpose: Shared pytest setup. Database tests are skipped automatically when MySQL is not reachable.
Imports: pathlib, pytest, sys, uuid
Functions:
- unique_phone()
- unique_email()
- require_database()
- customer_id(require_database) - A temporary customer that is removed (with its accounts/transactions) after the test.
- account_id(customer_id) - A SAVINGS account with an opening balance of 1000.

Notes for review
- Member 1 owns backend services, utilities, tests for transactions/validators, requirements and final merge.
- Member 2 owns database schema, sample data and database tests.
- Member 3 owns analysis modules and analysis tests.
- Member 4 owns Streamlit dashboard, pages, charts and login/landing UI.
- Shared files (docs, README, tests helpers) are reviewed by all members.

- The final ZIP excludes .venv, .env and caches; .env must be created locally from .env.example.